



**UNIVERSITATEA
TEHNICĂ
DIN CLUJ-NAPOCA**

Documentatie Proiect LFT

Limbaje Formale si Translatoare

Autor: Mirisan Octavian

Grupa: 30231

FACULTATEA DE AUTOMATICA
SI CALCULATOARE

6 Iunie 2025

Cuprins

1	Introducere	2
1.1	Scopul proiectului	2
2	Descriere teoretică	2
2.1	Structura JSON	2
2.2	Structura XML	2
3	Implementare	3
3.1	Lex	3
3.2	Yacc	4
4	Testare și validare	5
4.1	Cazuri acoperite	5
4.2	Limitări	6
5	Concluzii	7
5.1	Ce am învățat din acest proiect	7
5.2	Posibilități de dezvoltare ulterioară	7

1 Introducere

1.1 Scopul proiectului

Scopul acestui proiect este realizarea unui program care transformă un fișier JSON într-un fișier XML echivalent, păstrând structura și conținutul inițial.

Pentru implementare am folosit Lex pentru analiza lexicală și Yacc pentru analiza sintactică și generarea rezultatului. Programul citește fișierul JSON de intrare, îl parsează și generează automat un fișier `output.xml` cu datele convertite.

Proiectul mi-a oferit ocazia să aplic noțiuni de gramatici, analiză sintactică și lucru cu structuri arborescente, într-un context practic, ușor de testat și extins.

2 Descriere teoretică

2.1 Structura JSON

JSON (JavaScript Object Notation) este un format text simplu folosit pentru stocarea și schimbul de date. Este structurat pe bază de perechi cheie-valoare și suportă tipuri de date precum: stringuri, numere, boolean (`true/false`), `null`, array-uri și obiecte imbricate.

Un obiect JSON începe și se termină cu acolade `{}`, iar fiecare cheie este un string urmat de caracterul `:`, urmat de o valoare. Multiple perechi sunt separate prin virgule. Array-urile sunt liste de valori între paranteze drepte `[]`.

Exemplu:

```
{
  "nume": "Maria",
  "varsta": 22,
  "note": [10, 9, 8.5],
  "activ": true
}
```

În proiect am tratat toate tipurile de date de bază, inclusiv obiecte și array-uri imbricate.

2.2 Structura XML

XML (eXtensible Markup Language) este un format utilizat pentru reprezentarea ierarhică a datelor. Spre deosebire de JSON, care este concis, XML folosește taguri de deschidere și închidere pentru a marca fiecare element.

Fiecare valoare este încapsulată într-un tag cu numele său, de forma `<eticheta>valoare</eticheta>`. Structura este arborescentă, ceea ce îl face potrivit pentru reprezentări bine delimitate.

Exemplu echivalent în XML:

```
<nume>Maria</nume>
<varsta>22</varsta>
<note>
  <item>10</item>
  <item>9</item>
  <item>8.5</item>
</note>
<activ>true</activ>
```

3 Implementare

3.1 Lex

Fișierul `proiect.1` conține specificația pentru analizorul lexical al programului. Scopul său este să identifice corect tokenii din fișierul JSON și să-i transmită mai departe către parserul Yacc.

Am definit reguli pentru recunoașterea următoarelor tipuri de tokeni:

- delimitatori JSON: `{`, `}`, `[`, `]`, `:`, `,`
- stringuri (valori și chei): expresie regulată care detectează textul între ghilimele
- numere: atât întregi, cât și zecimale
- booleani: `true`, `false`
- `null`

Pentru toate valorile de tip string, număr, boolean și null, am folosit același token, `STRING`, pentru a simplifica partea sintactică. Valoarea asociată tokenului este stocată în `yylval.str`, folosind `strdup()` pentru alocare dinamică.

Exemplu de regulă:

```
\("[^"]")*\\" {
    yylval.str = strdup(yytext + 1);
    yylval.str[strlen(yyval.str) - 1] = '\\0';
    return STRING;
}
```

Această regulă elimină ghilimelele de la început și sfârșit și transmite valoarea ca `char*` către Yacc.

De asemenea, am ignorat spațiile, tab-urile și newline-urile, iar pentru caracterele neprevăzute am introdus un mesaj de eroare cu `printf`.

Lexerul a fost conectat cu parserul prin includerea fișierului `proiect.tab.h`.

3.2 Yacc

Fișierul `proiect.y` definește partea sintactică a aplicației, folosind Yacc (Bison). Rolul acestuia este să construiască structura XML pe baza tokenilor trimiși de Lex, folosind reguli gramaticale recursive care reflectă structura JSON.

Simbolul de start este `json`, care așteaptă un obiect JSON complet. Output-ul rezultat este scris automat într-un fișier numit `output.xml`, utilizând funcții C clasice (`fopen`, `fprintf`, `fclose`).

Am definit tokenii și non-terminalii folosind un `%union`, cu tipul `char*` pentru toate valorile, întrucât totul este convertit în text în formatul XML. Tokenul principal folosit pentru chei și valori este `STRING`, pentru simplitate.

Reguli importante din parser:

- **object** – corespunde unei secvențe de perechi cheie-valoare, delimitate de acolade.
- **pair** – transformă o pereche JSON de forma `"cheie": valoare` într-un tag XML: `<cheie>valoare</cheie>`.
- **value** – poate fi un string, un obiect imbricat sau un array.
- **array** – este tratat ca o secvență de elemente convertite în taguri `<item>`.

Am folosit concatenări de stringuri cu `malloc`, `strcpy`, `strcat` și `snprintf` pentru a construi outputul XML pas cu pas, fără a folosi biblioteci externe.

Exemplu de regulă:

```
pair: STRING COLON value {
    size_t size = strlen($1) * 2 + strlen($3) + 20;
    $$ = malloc(size);
    snprintf($$, size, "<%s>%s</%s>\n", $1, $3, $1);
    free($1); free($3);
}
```

Array-urile sunt tratate recursiv, fiecare element fiind introdus într-un tag `<item>`. De asemenea, parserul suportă obiecte imbricate la orice nivel.

La final, rezultatul este inclus într-un tag de rădăcină `<root>` și scris în fișierul de ieșire. Codul este modular și poate fi extins ușor pentru a trata alte structuri sau validări suplimentare.

4 Testare și validare

4.1 Cazuri acoperite

Pentru validarea funcționalității, am folosit mai multe fișiere JSON care acoperă diferite structuri și tipuri de date. Scopul a fost să testez atât conversia de bază, cât și suportul pentru elemente mai complexe (array-uri, obiecte imbricate, booleani, null etc.).

- **Test 1 – JSON simplu, flat:** Fișier cu perechi cheie-valoare unde valorile sunt stringuri, numere și booleani.
- **Test 2 – JSON imbricat:** Obiect JSON care conține un alt obiect și array-uri cu valori numerice.
- **Test 3 – Array-uri de stringuri:** Verifică tratarea listelor de texte în interiorul unui obiect.
- **Test 4 – Combinare de structuri:** Obiect complex care include toate cele de mai sus, pentru a verifica transformarea recursivă completă.

```
{
  "nume": "Ana Popescu",
  "varsta": 30,
  "oras": "Cluj-Napoca",
  "email": "ana.popescu@example.com",
  "student": "true",
  "media": 9,
  "facultate": "Informatica"
}
```

Figura 1: Exemplu JSON simplu

```
<root>
  <nume>Ana Popescu</nume>
  <varsta>30</varsta>
  <oras>Cluj-Napoca</oras>
  <email>ana.popescu@example.com</email>
  <student>true</student>
  <media>9</media>
  <facultate>Informatica</facultate>
</root>
```

Figura 2: Rezultat XML pentru JSON simplu

```

{
  "student": {
    "nume": "Maria Pop",
    "varsta": 22,
    "note": [9.5, 8.75, 10],
    "discipline": ["Matematica", "Fizica", "Informatica"],
    "contact": {
      "email": "maria.pop@example.com",
      "telefon": "0712345678"
    },
    "finalizat": false,
    "lucrare": null
  }
}

```

Figura 3: Exemplu JSON imbricat cu array-uri

```

<root>
  <student> <nume>Maria Pop</nume>
  <varsta>22</varsta>
  <note><item>9.5</item>
<item>8.75</item>
<item>10</i></note>
  <discipline><item>Matematica</item>
<item>Fizica</item>
<item>Informatica</i></discipline>
  <contact> <email>maria.pop@example.com</email>
  <telefon>0712345678</telefon>
</contact>
  <finalizat>false</finalizat>
  <lucrare>null</lucrare>
</student>
</root>

```

Figura 4: Rezultat XML pentru JSON complex

4.2 Limitări

Deși implementarea acoperă majoritatea structurilor uzuale din JSON, există câteva limitări:

- Nu se realizează indentare în output-ul XML (fișierul rezultat este pe linii separate, dar fără spațiere pe niveluri).
- Nu există validare de sintaxă XML (de exemplu, nu se verifică caractere speciale care ar trebui escape-uite).
- Nu este tratată ordinea elementelor într-un mod garantat (dar în JSON oricum nu este relevantă).
- Fișierele JSON foarte mari pot necesita optimizări suplimentare (memorie și performanță).

5 Concluzii

5.1 Ce am învățat din acest proiect

Acest proiect mi-a oferit ocazia să înțeleg mai bine cum funcționează Lex și Yacc în practică, cum pot fi definite reguli gramaticale pentru a parsa un format complex precum JSON și cum se construiește un output structurat, cum este XML. De asemenea, am învățat să lucrez cu alocare dinamică de memorie și manipulare de stringuri în C, într-un context aplicat.

5.2 Posibilități de dezvoltare ulterioară

Printre direcțiile în care proiectul poate fi extins se numără:

- suport pentru array-uri de obiecte
- escape corect pentru caractere speciale în XML (&, <, etc.)
- validare automată a structurii XML generate
- conversia inversă: XML $\rightarrow$ JSON
- interfață grafică simplă sau rulare în browser